



AI / ML Engineering

Hiring Challenge — Problem Statements

June 2026

Prepared by: Aivar Innovations

Overview

Candidates are tasked with building production-grade AI agents — autonomous systems capable of making decisions, executing tasks, and delivering reliable output for real-world use cases.

Pick one or more of the four agents below and implement them to demonstrate your ability to design, build, and ship an agent that actually works. Each agent is structured across three progressive levels of complexity: start with Level 1 and advance only when that level is solid.

What Does 'Production-Grade Agent' Mean?

A production-grade agent is autonomous but not reckless. It must exhibit all five of the following traits:

1. Make decisions — picks the right action based on context, not a hardcoded script
2. Validate aggressively — verifies its own output, detects hallucinations, and catches edge cases
3. Fail clearly — explains why it failed, what it tried, and what it needs from the operator
4. Maintain guard rails — knows its scope and operates strictly within it
5. Be observable — every action, decision, and confidence level is logged and traceable

Evaluation Criteria

All submissions are assessed across five dimensions regardless of which agent is chosen:

Criterion	What We Look For
Code Quality & Architecture	Well-structured, maintainable code with sound design patterns and clear separation of concerns
Real-World Complexity	Handles edge cases, API failures, and unexpected inputs without breaking or silently failing
Agentic Design Thinking	Clear decision logic, appropriate guard rails, and fully observable reasoning at every step
Completeness	A finished, demonstrably working deliverable — not a half-built proof-of-concept
Production Readiness	Deployable by another engineer with minimal changes; documented, tested, and not reliant on your laptop

Available Agents

Choose one or more of the following agents. All four are equal in weight — a complete Level 1 on any agent is valued over an incomplete attempt at all four.

Agent 1 Design Audit Agent	Agent 2 Offensive Security Agent	Agent 3 Discovery Agent	Agent 4 AWS Cost Optimization Agent
--------------------------------------	--	-----------------------------------	---

Agent 1: Design Audit Agent

When developers make frontend changes, there is no fast way to determine if the UI improved or degraded from a design perspective. Code reviewers focus on logic, not visual quality. Designers cannot review every pull request. This agent automatically evaluates UI/UX quality changes and provides actionable feedback — acting as an automated design reviewer embedded in the development pipeline.

Why It Matters

- **Design issues are invisible to code review.** Colors may look fine but fail WCAG contrast requirements. Spacing may be off but near-impossible to detect in a diff.
- **Designers cannot scale manually.** By the time a human reviews, bad changes may already be in production and compounding across subsequent PRs.
- **Speed drives quality.** Design feedback delivered in seconds — not days — allows developers to resolve issues while context is still fresh.
- **Accessibility is a legal requirement.** Contrast failures, small tap targets, and missing focus states are not just UX concerns — they create compliance and litigation risk.

What "Production-Ready" Means for This Agent

No hallucinated findings — every finding must reference something visible in the provided image, not an assumption.

Specific over vague — 'primary CTA is same weight as secondary actions, making priority unclear' beats 'bad hierarchy'.

Handles edge cases — dark mode, responsive breakpoints, partially loaded states, and dense data tables.

Structured output — JSON with resource identifier, principle violated, severity, location, recommendation, and confidence score.

At Level 3, operates fully autonomously without a human in the loop for each capture.

Progressive Levels

Level	What to Build	Key Challenge
Level 1	Accept a screenshot and identify design issues across five principles: Visual Hierarchy, Contrast (WCAG AA), Spacing, Alignment, and Consistency	No hallucinations — every finding must be visible and precisely located in the image
Level 2	Accept two screenshots and classify each visual difference as improvement, regression, or neutral with confidence scoring	Distinguish subjective aesthetic changes from objective regressions — confidence scores must reflect real uncertainty
Level 3	Log in to a live website, navigate to key pages, capture screenshots, compare against stored baselines, and report regressions autonomously	Filter dynamic content (timestamps, session IDs, loading states) from real regressions without false positives or silent skips

Level 1 — Single Page Design Analysis

The agent accepts a single screenshot (PNG, JPG, or WebP) and systematically evaluates it against five design principles. Each finding must identify what the issue is, where it appears on the page, why it matters to users, and a specific recommendation to fix it — not vague directional advice.

Acceptance Criteria

☐	Accepts image input: PNG, JPG, and WebP
☐	Identifies a minimum of 3 distinct design issues per screenshot
☐	Each finding includes: principle violated, location on page, user impact, and specific recommendation
☐	Returns structured JSON output with confidence scores per finding (0–100%)
☐	Severity classification per finding: critical / high / medium / low / info
☐	Zero hallucinations — all findings must be verifiable in the provided image

Level 2 — Before/After Regression Analysis

The agent accepts two screenshots — a baseline and a current state — and produces a structured diff. For each detected change, it explains what changed, where, the direction of change (improvement / regression / neutral), the user experience impact, and a confidence score. Hex color values and pixel measurements are included wherever measurable.

Acceptance Criteria

☐	Accepts two images: baseline (before) and current (after)
☐	Detects a minimum of 5 visual differences with specific details (hex values, pixel measurements where applicable)
☐	Classifies each difference: improvement / regression / neutral with written reasoning
☐	Explains the user experience impact of each change
☐	Includes a per-finding confidence score (0–100%)
☐	Flags accessibility regressions explicitly: contrast ratio drops, font size reductions, spacing compression
☐	Produces an overall verdict with a summary of net improvement vs. net regression

Level 3 — Autonomous UI/UX Regression Testing

The agent operates end-to-end without human intervention: it authenticates to a target website, navigates a configured set of pages, captures consistent screenshots, compares them against a stored baseline, and produces a regression report. The key engineering challenge is distinguishing dynamic content that changes on every load from genuine visual regressions that indicate a bug or bad change.

Acceptance Criteria

☐	Authenticates to a website autonomously — handles login forms, credential injection, and redirects
☐	Navigates to a minimum of 3 configured pages without user prompting

☐	Captures clean, consistent screenshots at a fixed viewport and scroll state
☐	Maintains a versioned baseline store — first run creates baseline, subsequent runs compare against it
☐	Identifies real regressions: layout breaks, color shifts, spacing changes, unreadable text
☐	Filters false positives: ignores timestamps, counters, loading spinners, and session-specific tokens
☐	Confidence score on every finding with evidence (pixel diff percentage, affected region)
☐	Completes a full scan cycle in under 3 minutes for 3–4 pages
☐	Supports explicit baseline refresh after a change has been reviewed and approved
☐	Produces a human-readable report alongside the structured JSON output

Agent 2: Offensive Security Agent

With infrastructure shipping daily, manual security reviews happen at best quarterly. Between reviews, misconfigurations and vulnerabilities silently accumulate. Security engineers spend time on routine checks that should be automated. This agent continuously scans infrastructure, code, and running systems for security issues — acting as a paranoid auditor that never sleeps.

Why It Matters

- **Security issues accumulate silently.** New code ships, configurations drift, and credentials leak — often unnoticed until a breach occurs or a compliance audit fails.
- **Manual reviews are expensive and infrequent.** A quarterly security review cannot catch a misconfiguration introduced on day two of the quarter. Continuous automated scanning closes that gap.
- **Most checks are mechanical and automatable.** Port exposure, IAM key rotation, S3 bucket ACLs, and known CVEs follow predictable patterns that do not require expert intuition to detect.
- **Speed prevents incidents.** Security warnings delivered while code is being written are far more effective than post-deployment alerts. The earlier the signal, the lower the remediation cost.

What "Production-Ready" Means for This Agent

No false positives on Critical findings — a Critical severity rating must be backed by evidence, not pattern-match ambiguity.

Actionable remediation — not 'you have a vulnerability' but 'package X version Y has CVE-2024-123, the fix is to upgrade to Y+1'.

Handles scale — scanning 50 resources should not take 30 minutes; parallelism and rate-limit handling are expected.

Knows what it cannot access — when a permission is denied or an API is unreachable, the agent reports it explicitly rather than assuming the resource is clean.

At Level 3, runs continuously on a schedule without human intervention and escalates Critical findings immediately.

Progressive Levels

Level	What to Build	Key Challenge
Level 1	Read a YAML/JSON security checklist and execute each check against real AWS infrastructure	Zero false positives on Critical severity — unsubstantiated ratings destroy trust in the tool
Level 2	Extend coverage to three domains: AWS infrastructure, running application APIs, and code dependency CVEs	Each domain has different signal norms — prioritize by business impact, not by which domain produced the finding
Level 3	Run scans on a configurable schedule, deduplicate across runs, escalate Critical	Silent scan failures are worse than missed scans — the

	findings, and track remediation SLAs without human intervention	agent must report its own health and never assume success
--	---	---

Level 1 — Checklist-Based Infrastructure Scanning

The agent reads a security checklist configuration file (YAML or JSON) defining a set of checks to run, executes each check against real infrastructure using the AWS CLI or SDK, and produces a structured findings report. Every finding must include the affected resource, severity level, raw evidence from the API, business impact, and a specific remediation command the operator can run immediately.

Acceptance Criteria

☐	Reads a security checklist configuration file in YAML or JSON format
☐	Executes 10 or more distinct checks: IAM policies, S3 ACLs, security groups, encryption at rest, MFA status
☐	Each finding includes: resource ARN, severity, raw evidence from the API, business impact, and step-by-step remediation
☐	Handles API errors gracefully — 403, 429, and timeouts are logged, not silently swallowed
☐	Produces both structured JSON (machine-readable) and Markdown (human-readable) reports
☐	Zero false positives on Critical severity findings
☐	All checks execute against real infrastructure — no mocked API responses

Level 2 — Multi-Domain Scanning

The agent expands beyond infrastructure to scan running application endpoints (authentication, CORS, rate limiting, security headers), parse dependency manifests for known CVEs, and detect hardcoded credentials across repositories and config files. Findings across all domains are merged, deduplicated, and ranked by estimated business impact — a leaked database credential ranks above a missing security header regardless of which scanner caught it.

Acceptance Criteria

☐	Scans a minimum of 3 domains: AWS infrastructure, live API endpoints, and code dependencies
☐	Infrastructure: 10 or more checks covering IAM, S3, security groups, encryption, and unused resources
☐	APIs: 5 or more checks covering authentication bypass, CORS misconfiguration, rate limiting, security headers, and error message disclosure
☐	Dependencies: accurate CVE detection with package name, installed version, CVE ID, CVSS score, and fix version
☐	Secrets scanning: detects AWS access keys, API tokens, and common password patterns in repositories and config files
☐	Cross-domain deduplication — the same vulnerability found by two scanners appears once

- | | |
|--------------------------|--|
| ☐ | False positive rate below 5% on High and Critical severity findings |
| ☐ | Findings ranked by business impact — not alphabetically or by domain |

Level 3 — Autonomous Continuous Scanning

The agent operates as a persistent service: scans run on a configured schedule, findings are stored with timestamps and lifecycle state, new findings trigger notifications, and re-appearing findings update their status rather than creating duplicates. Critical findings escalate immediately to a configured channel. The agent tracks whether findings are remediated within SLA and alerts on breaches. Scan health is reported explicitly — if a check fails, the failure is surfaced, not silently skipped.

Acceptance Criteria

- | | |
|--------------------------|--|
| ☐ | Runs full scans (infrastructure + APIs + dependencies) on a configurable schedule with no manual trigger |
| ☐ | Stores findings with a full lifecycle: opened, updated, resolved, re-opened, and SLA status |
| ☐ | Deduplicates across scan runs — a recurring finding updates its record, not adds a new one |
| ☐ | Escalates Critical findings to a configured notification channel immediately upon detection |
| ☐ | SLA tracking: alerts when a Critical finding remains unresolved beyond 24 hours |
| ☐ | Audit trail: all scans, findings, actions, and status changes are logged with actor and timestamp |
| ☐ | Trend reporting: security posture score over time, showing whether the environment is improving or degrading |
| ☐ | Scan health reporting: if a check fails or a permission is denied, the failure is logged and surfaced — never silently skipped |
| ☐ | Optional safe auto-remediation for low-risk findings with explicit logging and a human-approval gate for anything destructive |

Agent 3: Discovery Agent

Enterprise customers operate dozens of systems — Salesforce, NetSuite, Coupa, data warehouses, and more — but rarely maintain an accurate inventory. When they want to automate workflows, they spend weeks mapping systems and planning integrations before writing a line of code. This agent reads through unstructured company documents, discovers all systems in use, identifies integration gaps, and generates working connector code.

Why It Matters

- **No enterprise has a complete system map.** Information is scattered across PDFs, wikis, old emails, and tribal knowledge that leaves the moment someone changes jobs.
- **Integration planning fails without inventory.** You cannot plan what to automate if you do not know what systems you have. Incomplete discovery causes mid-build surprises and costly rework.
- **Integration development is repetitive by design.** Connecting System A to System B follows the same pattern every time: authenticate, paginate, handle errors, retry. It should be generated, not hand-coded from scratch.
- **Confidence matters as much as coverage.** An agent that invents systems it did not find is worse than one that finds fewer systems. Uncertainty must be flagged, not hidden.

What "Production-Ready" Means for This Agent

Handles incomplete information — a PDF mentioning 'Salesforce' with no further context should still be captured with an appropriate confidence score.

No hallucinations — the agent must never invent systems, integrations, or data flows that are not present in the source documents.

Flags uncertainty explicitly — a confidence score below 70% must surface a note explaining what was inferred and what was missing.

Generated code is 70–80% production-ready — an engineer reviewing it should find fewer than 10% of lines requiring significant changes.

At Level 3, generated connectors handle real API complexity: authentication, pagination, rate limiting, retries, and error handling — not just the happy path.

Progressive Levels

Level	What to Build	Key Challenge
Level 1	Process a mixed-format document collection (PDFs, images, wikis, spreadsheets) and extract a structured system inventory	Handle incomplete information — infer confidently where evidence is strong, flag uncertainty honestly where it is not
Level 2	Map a list of automation use cases to the systems they require, identify missing integrations, and prioritize gaps by business impact	Understand business value — a workflow that runs 1,000 times a year matters more than one that runs 5 times, regardless of technical complexity

Level 3	For each identified gap, generate a working connector and an agent definition that can execute the end-to-end workflow	Generated code must be 70–80% production-ready — an engineer reviewing it should find fewer than 10% of lines requiring significant changes
----------------	--	---

Level 1 — Systems Discovery from Knowledge Base

The agent processes a collection of documents in mixed formats and extracts a structured inventory of every system found. For each system, it captures name, category, authentication method, key data entities, business processes it supports, criticality to operations, and a confidence score reflecting how much evidence the extraction is based on. Systems mentioned once in passing are flagged for manual review rather than asserted with high confidence.

Acceptance Criteria

☐	Processes multiple document types: PDF, image, plain text, Markdown, and spreadsheet
☐	Extracts 80% or more of all systems mentioned across the document collection
☐	For each system: name, category, auth method, key entities, business processes, and criticality
☐	Confidence score on each extraction: 95%+ for explicit mentions, 70–90% for inferred, below 70% flagged for human review
☐	Flags uncertain findings with a note on what was inferred and what evidence was missing
☐	Produces structured JSON output with source document reference for each extracted system
☐	Zero hallucinations — does not invent systems absent from the source documents

Level 2 — Use Case Mapping & Integration Gap Analysis

Given a system inventory (from Level 1 or provided) and a list of automation goals, the agent maps each use case to the systems involved, traces the data flows between them, checks which integrations already exist versus which are missing, and produces a prioritized gap list. Priority reflects the combination of how many use cases a gap blocks, how frequently those workflows run, and the estimated effort to close the gap.

Acceptance Criteria

☐	Maps 5 or more use cases to their involved systems correctly — no missed system dependencies
☐	Traces data flows for each use case: source system, destination system, entity type, and trigger event
☐	Identifies integration gaps accurately: marks each required integration as available or missing
☐	Estimates implementation effort for each missing integration (rough order-of-magnitude acceptable)
☐	Prioritizes gaps by business impact: use case frequency, criticality, and number of downstream use cases blocked

☐	Produces dependency mapping: 'Integration X must exist before use case Y can be automated'
☐	Output is structured JSON consumable by a downstream planning or ticketing tool

Level 3 — Integration & Agent Code Generation

The agent generates two artifacts per integration gap: a working connector module (Python or Node.js) implementing authentication, CRUD operations, pagination, rate limiting, retries, and error handling; and an agent definition (YAML) specifying the system prompt, available tools, step-by-step workflow, and test scenarios. The generated code is not pseudocode — it must be importable, runnable, and structured for an engineer to review and deploy with minimal rework.

Acceptance Criteria

☐	Generates working integration code — not pseudocode, stubs, or illustrative examples
☐	Each connector includes: authentication, CRUD operations, pagination, rate limiting, retries, and error handling
☐	Generated code is importable and runnable without a complete rewrite
☐	Includes unit tests covering authentication, CRUD operations, error conditions, and edge cases
☐	Generated agent definitions include: system prompt, tool bindings, workflow steps, and test scenarios
☐	Code follows consistent conventions: naming, file structure, and error handling patterns
☐	Includes a setup README and dependency manifest (requirements.txt or package.json) per generated artifact
☐	70–80% production-ready: an engineering review should find fewer than 10% of lines requiring significant changes
☐	Handles real API complexity: non-standard auth flows, cursor-based pagination, and transient failure retries

Agent 4: AWS Cost Optimization Agent

Cloud infrastructure costs grow faster than teams can review them. Manual audits happen infrequently, waste accumulates silently, and by the time a spike is noticed the damage is already on the bill. This agent acts as a continuous cost intelligence layer — observing what is being created and consumed across AWS accounts, classifying findings by risk and savings potential, and converting raw AWS data into clear, accountable actions.

Why It Matters

- **Waste accumulates silently.** Idle EC2 instances, unattached EBS volumes, stale snapshots, and forgotten NAT-heavy dev environments rarely trigger alerts — they just drain budgets month after month.
- **Attribution is broken without tag governance.** Spend without owner, customer, or environment tags cannot be routed to anyone. An agent that enforces tagging compliance continuously creates the foundation for every other optimization.
- **Anomaly detection needs context to be actionable.** A 40% cost spike means nothing without knowing which account, service, and customer it belongs to. Catching the signal is step one; the agent provides the context that makes it actionable.
- **Human review time is the bottleneck, not data.** Trusted Advisor, Compute Optimizer, and Cost Explorer already surface findings programmatically. The agent's job is to triage that signal into a ranked, human-reviewable action list.

What "Production-Ready" Means for This Agent

No hallucinated findings — every finding must cite the specific API, resource ARN, and account ID that produced it.

Confidence scoring on every recommendation — a rightsize suggestion must include utilization percentile, observation window length, and a confidence level.

Safe by default — the agent never deletes, stops, or modifies a resource without an explicit pre-approved action. All automated actions are logged with actor, timestamp, and justification.

Observable at every step — the agent emits a structured log for every check: what it queried, what it found, how it classified the finding, and what action it proposed.

Multi-account aware — correctly attributes spend to the originating member account, not the payer, using cross-account assume-role patterns.

Progressive Levels

Level	What to Build	Key Challenge
Level 1	Execute a five-check daily audit: untagged spend, anomalies, idle resources, non-prod always-on, and top cost movers	Every finding must be traceable to a real resource with a specific ARN and evidence source — no inferred or assumed findings
Level 2	Extend coverage to an AWS Organizations account hierarchy; add confidence scoring, owner routing, and a prioritized savings report	Attribution must follow money to the correct member account — payer-level aggregation without

		account-level breakdown is not sufficient
Level 3	Run continuously on a schedule, enforce tagging policy, execute approved low-risk remediations, and surface trend data on cost posture over time	Automated actions must be safe, reversible, and auditable — silent or irreversible changes are worse than no automation at all

Level 1 — Daily Cost Audit

The agent connects to one or more AWS accounts via read-only IAM role and runs five targeted checks on a daily cadence: untagged resources with nonzero spend, new cost anomalies since the last run, idle or underutilized compute and storage, non-prod resources running outside expected hours, and the top 20 cost movers by day-over-day delta. Each finding is classified into an action bucket (delete / schedule / rightsize / tag-fix / needs-human-review) and ranked by estimated monthly savings.

Acceptance Criteria

☐	Connects to real AWS accounts via read-only IAM role — no hardcoded credentials
☐	Executes all five daily audit checks against live data — not mocked or stubbed responses
☐	Untagged resources: identifies resources missing owner, environment, or customer tags that incurred spend in the last 7 days
☐	Anomalies: surfaces Cost Anomaly Detection findings since the last run with account, service, and day-over-day delta
☐	Idle resources: queries Trusted Advisor and Compute Optimizer findings, each with resource ID, owner tag, and recommended action
☐	Non-prod always-on: identifies dev/test/staging resources running outside approved hours with suggested stop schedules
☐	Top cost movers: lists top 20 resources or services by absolute spend increase in the last 24 hours
☐	Each finding classified: delete / schedule / rightsize / tag-fix / needs-human-review with reasoning
☐	Findings ranked by estimated monthly savings — highest-impact items first
☐	Produces both structured JSON and a human-readable digest suitable for Slack or email

Level 2 — Multi-Account Triage & Recommendations

The agent operates across an AWS Organizations hierarchy, using cross-account assume-role to collect findings from all member accounts. Findings are enriched with owner tags, account metadata, and cost allocation category. The triage layer classifies each finding with a confidence score based on observation window length and data quality, and routes findings to the correct owner via a ticketing or notification integration. The output is a weekly savings report ranked by estimated annual impact.

Acceptance Criteria

☐	Operates across multiple accounts using AWS Organizations or an explicit account list with cross-account assume-role
☐	Correctly attributes each finding to its originating member account — not the payer
☐	Enriches findings with owner tag, account alias, cost allocation category, and environment
☐	Confidence score per finding: low (less than 7 days observation), medium (7–30 days), high (30+ days)
☐	Routes findings to the correct owner via Slack, Teams, or a ticketing system (Jira, ServiceNow)
☐	Deduplicates findings across accounts — the same pattern in multiple accounts appears as related findings, not duplicates
☐	Produces a weekly savings report: total estimated annual savings, breakdown by action type, and top 10 opportunities
☐	Handles missing permissions gracefully — reports which accounts or services were inaccessible rather than silently skipping

Level 3 — Autonomous Cost Governance

The agent operates as a persistent governance service: scheduled scans feed a findings database, tag policy violations trigger automated correction workflows, approved low-risk actions (stop schedules, TTL tag enforcement) execute without human intervention, and an immutable audit log records every action. Cost posture is tracked as a score over time, with weekly trend reports surfacing whether the environment is improving or degrading. All automated actions require pre-approval configuration and are reversible where the AWS API supports it.

Acceptance Criteria

☐	Runs full audits on a configurable schedule without manual triggers
☐	Enforces tag policy: automatically opens a correction ticket or applies missing tags per a pre-approved tag policy definition
☐	Executes approved low-risk remediations: stop schedules on past-TTL resources, S3 lifecycle rule attachment, log group retention correction
☐	Immutable audit log: every action records resource ARN, account, actor (agent or human), timestamp, action taken, and justification
☐	No destructive or irreversible action without explicit human approval in the configuration
☐	Cost posture score tracked over time: weekly trend with explanation of what drove improvement or degradation
☐	SLA tracking: alerts when a high-severity finding remains unactioned beyond a configured threshold
☐	Scan health reporting: surfaces which checks failed, which accounts were unreachable, and why — never silently skips
☐	Produces a monthly executive summary: total spend saved, findings resolved, findings opened, and posture trend

Submission Guidelines

How to Approach the Challenge

1. Pick one or more agents — your choice
2. Start with Level 1 and ship something complete before advancing
3. Advance to Level 2 or 3 only if Level 1 is solid and working
4. Be honest about trade-offs — clear thinking is valued over heroics
5. Write clean code — assume another engineer will maintain it

Common Pitfalls to Avoid

Do not try to build all levels simultaneously — you will finish nothing well

Do not over-engineer v1 — apply YAGNI (you aren't gonna need it)

Do not ignore edge cases that will surface in real usage

Do not treat LLM output as ground truth — validate, verify, and ask for confirmation

Required Deliverables

Deliverable	Description
Video Recording	Explain your solution with a demo and detailed walkthrough
Code	Zip file containing all source code
Write Up	Brief document covering agent architecture, key decisions, and trade-offs made

Final Evaluation Summary

Across all submissions, we are looking for candidates who demonstrate the following:

- **Autonomous system design.** Can you design a decision-making agent with appropriate guard rails and fully observable reasoning?
- **Production code quality.** Is your code clean, maintainable, and deployable by another engineer without significant rework?
- **Edge case thinking.** Do you proactively handle failures, unexpected inputs, and boundary conditions?
- **Completeness.** Did you ship something complete and demonstrably useful — not a skeleton with aspirational comments?

Good luck.

We look forward to reviewing your work.